USWAG
Unified System for Work and Growth · Performance Report

USWAG Performance Report: Execution Plans Before and After Indexing

Environment: MySQL 8.0.46, InnoDB. Scratch database with **50,000 Sales** and **100,000 Sale_Details** rows (generated with a recursive CTE, 200 sellers).

Query (seller cash-flow per sale, same pattern as `vw_seller_cash_flow` / `sp_*` reports):

```
SELECT sa.sale_id, SUM(sd.line_total) AS sale_total
FROM Sales sa JOIN Sale_Details sd ON sd.sale_id = sa.sale_id
WHERE sa.seller_id = 7 AND sa.status = 'Paid'
GROUP BY sa.sale_id;
```

Result

	Before (no secondary index)	After
Access path	Full table scan of Sale_Details (100,000 rows) + 100,000 PK probes into Sales	Covering index lookup on Sales, 250 index probes into Sale_Details
Actual time	**about 146 ms**	**about 2.5 ms**
Speed-up		**about 58x**

Raw EXPLAIN ANALYZE output

```
=== BEFORE INDEXING ===
EXPLAIN
-> Table scan on <temporary>  (actual time=146..146 rows=250 loops=1)
    -> Aggregate using temporary table  (actual time=146..146 rows=250 loops=1)
        -> Nested loop inner join  (cost=45019 rows=4992) (actual time=0.486..146
rows=500 loops=1)
            -> Table scan on sd  (cost=10073 rows=99847) (actual time=0.455..26.3
rows=100000 loops=1)
            -> Filter: ((sa.`status` = 'Paid') and (sa.seller_id = 7))  (cost=0.25
rows=0.05) (actual time=0.00107..0.00107 rows=0.005 loops=100000)
                -> Single-row index lookup on sa using PRIMARY (sale_id=sd.sale_id)
(cost=0.25 rows=1) (actual time=753e-6..788e-6 rows=1 loops=100000)
```

```
=== AFTER INDEXING ===
EXPLAIN
-> Group aggregate: sum(sd.line_total)  (cost=259 rows=517) (actual time=0.0695..2.49
rows=250 loops=1)
    -> Nested loop inner join  (cost=207 rows=517) (actual time=0.054..2.34 rows=500
loops=1)
        -> Filter: (sa.`status` = 'Paid')  (cost=26.3 rows=250) (actual
time=0.0352..0.116 rows=250 loops=1)
            -> Covering index lookup on sa using idx_sales_seller_status (seller_id=7,
status='Paid')  (cost=26.3 rows=250) (actual time=0.0334..0.0766 rows=250 loops=1)
        -> Index lookup on sd using idx_saledetails_sale (sale_id=sa.sale_id)
(cost=0.518 rows=2.07) (actual time=0.00813..0.00855 rows=2 loops=250)
```

Indexing justification

- `idx_sales_seller_status (seller_id, status)`: the WHERE clause filters equality on both columns; seller_id first (high selectivity), status second makes the lookup covering.
- `idx_saledetails_sale (sale_id)`: join column on the child side; turns a table scan into an index lookup.
- `uq_sales_seller_client (seller_id, client_id)`: UNIQUE index that doubles as the duplicate-sync guard for offline sales (idempotent upload).
- `idx_sales_sync (sync_status, sale_date)`: serves the offline queue (`vw_offline_queue`).
- `idx_saledetails_product (product_id)`: serves product sales history and category reports.
- Trade-off: each index adds write cost on INSERT; the set is kept small because USWAG is read-heavy for reports and write-light per seller.

© 2026 Bryan Jay Lumabas and Claire Pillo